SQL Topic (End-to-End)

1. Types of SQL Language
2. Datatypes in SQL
3. Tables(Columns, Rows)
4. Constraints:
    1. Primary Key
    2. Foreign Key
    3. Unique Key
    4. Not Null
    5. Check
    6. Default
5. SQL Operators
    1. Arithmetic Operators
    2. Comparison Operators
    3. Logical Operators
    4. Bitwise Operators
    5. Assignment Operators
    6. Miscellaneous Operators
6. SQL Functions
    1. Aggregate Functions
    2. Scalar Functions
    3. String Functions
    4. Date Functions
    5. Conversion Functions
7. Joins
    1. left join
    2. right join
    3. inner join
    4. full outer join
    5. cross join
    6. self join
8. Subqueries
    1. Single Row Subquery
    2. Multiple Row Subquery
    3. Correlated Subquery
    4. Nested Subquery
9. Temporary Tables
10. Views
    1. Simple View
    2. Complex View

This is a complete SQL roadmap that covers almost everything from beginner to advanced. Since explaining everything in one response would exceed the maximum response length, I'll start with Part 1 (Topics 1–6) in detail. I can then continue with Parts 2–4.

SQL End-to-End Guide (Part 1)

1. Types of SQL Language

SQL commands are divided into five categories.

Type	Purpose	Commands
DDL	Define database objects	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Manipulate data	INSERT, UPDATE, DELETE, MERGE
DQL	Retrieve data	SELECT
DCL	Control permissions	GRANT, REVOKE
TCL	Transaction management	COMMIT, ROLLBACK, SAVEPOINT

DDL (Data Definition Language)

Used for creating and modifying database objects.

CREATE

```
CREATE TABLE Employee
(
    EmpID INT,
    Name VARCHAR(100),
    Salary NUMERIC(10,2)
);
```

ALTER

```
ALTER TABLE Employee
ADD Department VARCHAR(50);
```

DROP

Deletes entire object.

```
DROP TABLE Employee;
```

TRUNCATE

Deletes all records.

```
TRUNCATE TABLE Employee;
```

Difference:

```
DELETE → Removes selected rows
TRUNCATE → Removes all rows
DROP → Removes entire table
```

DML (Data Manipulation Language)

INSERT

```
INSERT INTO Employee
VALUES (101,'Ashish',80000);
```

UPDATE

```
UPDATE Employee
SET Salary=90000
WHERE EmpID=101;
```

DELETE

```
DELETE FROM Employee
WHERE EmpID=101;
```

DQL

Retrieve records.

```
SELECT * FROM Employee;
```

DCL

Grant permissions.

```
GRANT SELECT ON Employee TO User1;

REVOKE SELECT ON Employee FROM User1;
```

TCL

COMMIT

Makes changes permanent.

```
COMMIT;
```

ROLLBACK

Undo changes.

```
ROLLBACK;
```

SAVEPOINT

```
SAVEPOINT A;

ROLLBACK TO A;
```

---

2. SQL Datatypes

Numeric

```
SMALLINT
INT
BIGINT
DECIMAL
NUMERIC
FLOAT
REAL
DOUBLE PRECISION
```

Example

```
Salary NUMERIC(10,2)
Age INT
```

Character

```
CHAR
VARCHAR
TEXT
```

Example

```
Name VARCHAR(100)
```

Difference

```
CHAR → Fixed length

VARCHAR → Variable length

TEXT → Unlimited length
```

Date & Time

```
DATE

TIME

TIMESTAMP

INTERVAL
```

Example

```
JoiningDate DATE
```

```
CreatedDate TIMESTAMP
```

Boolean

```
BOOLEAN
```

Example

```
IsActive BOOLEAN
```

Binary

```
BYTEA (PostgreSQL)

BLOB (MySQL)
```

3. Tables

A table consists of

```
Columns
Rows
```

Example

Employee Table

EmpID	Name	Salary
101	Ashish	80000
102	Rahul	90000

Columns

```
EmpID

Name

Salary
```

Rows

```
101 Ashish 80000

102 Rahul 90000
```

Create Table

```
CREATE TABLE Employee
(
    EmpID INT,
    Name VARCHAR(100),
    Salary NUMERIC(10,2)
);
```

---

4. Constraints

Constraints maintain data integrity.

Primary Key

Uniquely identifies every record.

```
Cannot be NULL

Cannot repeat
```

Example

```
CREATE TABLE Employee
(
EmpID INT PRIMARY KEY,
Name VARCHAR(50)
);
```

Foreign Key

Creates relationship.

Department Table

```
CREATE TABLE Department
(
DeptID INT PRIMARY KEY,
DeptName VARCHAR(50)
);
```

Employee

```
CREATE TABLE Employee
(
EmpID INT PRIMARY KEY,
DeptID INT,
FOREIGN KEY(DeptID)
REFERENCES Department(DeptID)
);
```

Unique Key

No duplicate values.

```
Email VARCHAR(100) UNIQUE
```

Difference

Primary Key

```
Only one

Cannot NULL
```

Unique

```
Multiple allowed

Allows one NULL (DB dependent)
```

NOT NULL

```
Name VARCHAR(50) NOT NULL
```

CHECK

```
Age INT CHECK(Age>18)
```

DEFAULT

```
CreatedDate TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
```

---

5. SQL Operators

Arithmetic

```
+

-

*

/

%
```

Example

```
SELECT Salary+1000
FROM Employee;
```

Comparison

```
=

<>

!=

>

<

>=

<=
```

Example

```
SELECT *
FROM Employee
WHERE Salary>50000;
```

Logical

```
AND

OR

NOT
```

Example

```
SELECT *
FROM Employee
WHERE Salary>50000
AND Department='IT';
```

Bitwise

```
&

|

^

<<

>>
```

Example

```
SELECT 10 & 7;
```

Result

```
2
```

Assignment

Mostly in SQL Server.

```
=
```

Example

```
SET @Salary=50000;
```

Miscellaneous

```
IN

BETWEEN

LIKE

IS NULL

EXISTS

ANY

ALL
```

IN

```
SELECT *
FROM Employee
WHERE Department
IN('IT','HR');
```

BETWEEN

```
WHERE Salary
BETWEEN 50000
AND 100000;
```

LIKE

```
WHERE Name
LIKE 'A%'
```

Wildcards

```
%

_
```

EXISTS

```
SELECT *
FROM Employee E
WHERE EXISTS
(
SELECT 1
FROM Department D
WHERE E.DeptID=D.DeptID
);
```


6. SQL Functions

Functions simplify calculations and data manipulation.

Aggregate Functions

Work on multiple rows.

```
COUNT

SUM

AVG
```

```
MIN

MAX
```

Example

```
SELECT COUNT(*)
FROM Employee;
```

```
SELECT SUM(Salary)
FROM Employee;
```

```
SELECT AVG(Salary)
FROM Employee;
```

Scalar Functions

Return one value.

Examples

```
UPPER()

LOWER()

ROUND()

ABS()

SQRT()
```

Example

```sql
SELECT UPPER(Name)
FROM Employee;
```

String Functions

```
LENGTH()

SUBSTRING()

CONCAT()

TRIM()

REPLACE()

POSITION()
```

Example

```sql
SELECT SUBSTRING(Name,1,3)
FROM Employee;
```

```sql
SELECT CONCAT(FirstName,' ',LastName)
FROM Employee;
```

Date Functions (PostgreSQL)

Current Date

```sql
SELECT CURRENT_DATE;
```

Current Timestamp

```
SELECT NOW();
```

Difference

```
CURRENT_DATE

Only Date
```

```
NOW()

Date + Time
```

Add Days

```
SELECT CURRENT_DATE+5;
```

Difference

```
AGE()

DATE_PART()

DATE_TRUNC()
```

Example

```
SELECT AGE(NOW(),JoiningDate)
FROM Employee;
```

Conversion Functions

Convert one datatype to another.

CAST

```
SELECT CAST(Salary AS INTEGER)
FROM Employee;
```

:: (PostgreSQL)

```
SELECT Salary::INTEGER
FROM Employee;
```

TO_CHAR

```
SELECT TO_CHAR(NOW(),'DD-MM-YYYY');
```

TO_DATE

```
SELECT TO_DATE('2026-07-14','YYYY-MM-DD');
```

Summary

Topic	Covered
SQL Language	✅
Datatypes	✅
Tables	✅
Constraints	✅
Operators	✅
Functions	✅

SQL End-to-End Guide (Part 2)

This part covers:

1. Joins
2. Subqueries
3. Temporary Tables
4. Views

---

7. JOINS

What is a Join?

A JOIN is used to combine records from two or more tables based on a related column.

Suppose we have two tables.

Employee

EmpID	Name	DeptID
101	Ashish	1
102	Rahul	2
103	Amit	3
104	Rohit	NULL

Department

DeptID	DeptName
1	IT
2	HR
4	Finance

Relationship

```
Employee.DeptID -----> Department.DeptID
```

Types of Joins

```
              JOINS
                |
    ------------------------
    |     |     |    |     |
INNER LEFT RIGHT FULL CROSS SELF
```

1. INNER JOIN

Returns only matching rows.

```sql
SELECT
    e.EmpID,
    e.Name,
    d.DeptName
FROM Employee e
INNER JOIN Department d
ON e.DeptID = d.DeptID;
```

Output

EmpID	Name	DeptName
101	Ashish	IT
102	Rahul	HR

Explanation

```
Employee          Department

1  ------------> 1

2  ------------> 2

3  X

NULL X
```

Only matching records are returned.

---

2. LEFT JOIN

Returns

- **All rows from Left table**
- **Matching rows from Right table**

```
SELECT
e.EmpID,
e.Name,
d.DeptName
FROM Employee e
LEFT JOIN Department d
ON e.DeptID=d.DeptID;
```

Output

EmpID	Name	DeptName
101	Ashish	IT
102	Rahul	HR
103	Amit	NULL
104	Rohit	NULL

Explanation

```
LEFT TABLE
Always Returned

Employee

101
102
103
104

Department only if match exists
```

---

3. RIGHT JOIN

Returns

- All rows from Right table
- Matching rows from Left table

```
SELECT
e.Name,
d.DeptName
FROM Employee e
RIGHT JOIN Department d
ON e.DeptID=d.DeptID;
```

Output

Name	DeptName
Ashish	IT
Rahul	HR
NULL	Finance

4. FULL OUTER JOIN

Returns

- All rows from both tables

```
SELECT
*
FROM Employee e
FULL OUTER JOIN Department d
ON e.DeptID=d.DeptID;
```

Output

Employee	Department
Ashish	IT
Rahul	HR
Amit	NULL
Rohit	NULL
NULL	Finance

5. CROSS JOIN

Produces Cartesian Product.

Formula

```
Rows = Table1 × Table2
```

Example

Employee

```
A
B
```

Department

```
IT
HR
```

Output

```
A IT

A HR

B IT

B HR
```

SQL

```
SELECT *
FROM Employee
CROSS JOIN Department;
```

6. SELF JOIN

A table joins itself.

Employee

EmpID	Name	ManagerID
1	CEO	NULL
2	Ashish	1
3	Rahul	1
4	Amit	2

Query

```
SELECT
e.Name Employee,
m.Name Manager
FROM Employee e
LEFT JOIN Employee m
ON e.ManagerID=m.EmpID;
```

Output

Employee	Manager
CEO	NULL
Ashish	CEO
Rahul	CEO
Amit	Ashish

Join Summary

Join	Returns
INNER	Matching rows only
LEFT	All left + matching right
RIGHT	All right + matching left
FULL	All rows from both
CROSS	Every combination
SELF	Table joined with itself

8. SUBQUERIES

What is a Subquery?

A query written inside another query.

```
Outer Query
    |
Sub Query
```

Syntax

```
SELECT ...
FROM ...
WHERE column =
(
SELECT ...
);
```

Types of Subqueries

```
Subquery
    |
-----------------------
```

```
|   |   |   |
Single
Multi
Correlated
Nested
```

1. Single Row Subquery

Returns only one value.

Example

Highest salary employee

```
SELECT *
FROM Employee
WHERE Salary=
(
SELECT MAX(Salary)
FROM Employee
);
```

Inner Query

```
SELECT MAX(Salary)
FROM Employee;
```

Suppose

```
95000
```

Outer query becomes

```
SELECT *
FROM Employee
WHERE Salary=95000;
```

2. Multiple Row Subquery

Returns multiple rows.

Example

Employees working in IT or HR.

```
SELECT *
FROM Employee
WHERE DeptID IN
(
SELECT DeptID
FROM Department
WHERE DeptName IN ('IT','HR')
);
```

Inner query returns

```
1

2
```

Outer query becomes

```
WHERE DeptID IN (1,2)
```


3. Correlated Subquery

Runs once for every row.

Employee

DeptID	Salary
1	40000
1	80000
1	90000
2	30000
2	35000

Find employees earning above department average.

```
SELECT *
FROM Employee e
WHERE Salary >
(
SELECT AVG(Salary)
FROM Employee
WHERE DeptID=e.DeptID
);
```

Execution

```
Employee 1

Average calculated

Employee 2

Average calculated

Employee 3

Average calculated
```

Runs repeatedly.

---

4. Nested Subquery

Subquery inside another subquery.

```
SELECT *
FROM Employee
WHERE DeptID=
(
SELECT DeptID
FROM Department
WHERE Location=
(
SELECT Location
FROM Office
WHERE OfficeID=10
)
);
```

Three queries are involved.

EXISTS

Returns TRUE if rows exist.

```
SELECT *
FROM Department d
WHERE EXISTS
(
SELECT 1
FROM Employee e
WHERE e.DeptID=d.DeptID
);
```


ANY

```
SELECT *
FROM Employee
WHERE Salary > ANY
(
SELECT Salary
FROM Employee
WHERE DeptID=2
);
```

Means greater than at least one salary.

ALL

```sql
SELECT *
FROM Employee
WHERE Salary > ALL
(
SELECT Salary
FROM Employee
WHERE DeptID=2
);
```

Means greater than every salary.

Subquery Summary

Type	Returns
Single Row	One value
Multiple Row	Multiple values
Correlated	Runs once per outer row
Nested	Subquery inside another subquery

---

9. TEMPORARY TABLES

What is a Temporary Table?

A table created only for the current session.

Useful for

- **Intermediate calculations**
- **Reporting**
- **ETL processing**
- **Large transformations**

Create Temporary Table

(PostgreSQL)

```
CREATE TEMP TABLE TempEmployee
(
EmpID INT,
Name VARCHAR(100)
);
```

Insert

```
INSERT INTO TempEmployee
VALUES
(101,'Ashish'),
(102,'Rahul');
```

Retrieve

```
SELECT *
FROM TempEmployee;
```

After session ends

```
Table automatically disappears.
```


Temporary vs Permanent

Feature	Temporary	Permanent
Visible to others	No	Yes
Auto Delete	Yes	No
Lifetime	Session	Forever

10. VIEWS

What is a View?

A View is a virtual table created using a SELECT query.

```
Table

↓

View

↓

User
```

Data is not stored separately (except materialized views).

Simple View

Uses one table.

```
CREATE VIEW EmployeeView AS
SELECT
EmpID,
Name,
Salary
FROM Employee;
```

Use

```
SELECT *
FROM EmployeeView;
```

Complex View

Uses joins, aggregates or multiple tables.

```
CREATE VIEW EmployeeDepartment AS
SELECT
e.Name,
d.DeptName,
e.Salary
FROM Employee e
JOIN Department d
ON e.DeptID=d.DeptID;
```

Another example

```
CREATE VIEW DepartmentSalary AS
SELECT
DeptID,
AVG(Salary) AverageSalary
FROM Employee
GROUP BY DeptID;
```

Materialized View (PostgreSQL)

Unlike a normal view, data is physically stored.

```
CREATE MATERIALIZED VIEW EmployeeSummary AS
SELECT
DeptID,
COUNT(*) Employees,
AVG(Salary) AvgSalary
FROM Employee
GROUP BY DeptID;
```

Read

```
SELECT *
FROM EmployeeSummary;
```

Refresh

```
REFRESH MATERIALIZED VIEW EmployeeSummary;
```

Normal View vs Materialized View

Feature	View	Materialized View
Stores data	No	Yes
Always latest	Yes	No (requires refresh)
Performance	Slower on large data	Faster for reporting
Disk Space	No extra storage	Consumes storage

Advantages of Views

- **Hide complex SQL**
- **Restrict access to sensitive columns**
- **Reuse common queries**
- **Simplify reporting**
- **Improve maintainability**

Disadvantages

- **Complex views can be slower**
- **Materialized views require refresh**
- **Not all views are updatable**

Interview Questions

1. LEFT JOIN vs INNER JOIN?

- **INNER JOIN:** Returns only matching rows.
- **LEFT JOIN:** Returns all rows from the left table and matching rows from the right table.

2. DELETE vs TRUNCATE vs DROP?

- **DELETE:** Removes selected rows; can be rolled back within a transaction.
- **TRUNCATE:** Removes all rows quickly; table structure remains.
- **DROP:** Removes the table and its data permanently.

---

3. Correlated Subquery vs JOIN?

- **Correlated Subquery:** Executes once for each row of the outer query; often less efficient.
- **JOIN:** Typically faster because the optimizer can process tables together.

---

4. View vs Materialized View?

- **View:** Stores only the query definition and always reflects current data.
- **Materialized View:** Stores the query result physically and must be refreshed to reflect changes.

---

5. When should you use a Temporary Table?

- Breaking complex queries into simpler steps.
- Storing intermediate ETL or reporting results.
- Improving readability and, in some cases, performance by reusing intermediate datasets.

This completes **Part 2**. The next section, **Part 3**, covers **Stored Procedures, User Defined Functions (UDFs), Triggers, Window Functions (ROW_NUMBER, RANK, DENSE_RANK), and Common Table Expressions (CTEs)**, which are frequently asked in SQL and PostgreSQL interviews.